JavaScript Promises and Async Programming

Course Overview

Course Overview

Hi everyone, my name is Nate Taylor, and welcome to my course, JavaScript Promises and Async Programming. I am a solution architect at Aviture in Omaha, Nebraska. JavaScript Promises are a powerful solution to working with asynchronous programming; however, they can also be intimidating to learn, but they don't have to be. In this course, we're going to cover everything you need to know to begin working with promises and Async/Await in JavaScript. Some of the major topics that we will cover include asynchronous programming, consuming and creating promises, and using Async/Await in JavaScript. By the end of this course, you'll know how to work with promise-based libraries, as well as how to wrangle asynchronous code by creating your own promises. Before beginning the course, you should be familiar with basic JavaScript syntax. I hope you'll join me on this journey to learn promises, with the JavaScript Promises and Async Programming course, at Pluralsight.

Understanding Promises

Understanding Promises

Newcomers to JavaScript often struggle with its asynchronous nature, particularly if they're coming from another programming language that doesn't have callbacks. The struggle is intensified when they hear about promises, because they're told that promises make things better, but they're not necessarily simple. Before digging into how promises can help, let's take a minute to look at a fairly common problem, fetching data, and how the fact that it's asynchronous throws a wrench into the whole system. The code you see on the screen makes two separate HTTP requests. This block of code might not be something you see every day, so let's take a second to walk through the highpoints. It starts by creating a new XMLHttpRequest, this object allows us to make requests against an API. It specifies the path of the request in the open function, it then has an onload function that you can think of as success. Finally, it has a send function, that is the entire block is set up, and then the request is actually sent. You'll notice there are actually two xhr requests. The first request is to the order statuses endpoint, and the second request is to the orders endpoint. Once the

data comes back from the orders call, which can be seen in the onload function of the xhr request, it looks up the order status for that particular order in the list of order statuses that was returned above. Switching over to my browser, I can run the code that you just saw. When I click the Race Condition, the results in the bordered area below will update. You can see that the value comes back as undefined. That could be for a variety of reasons, including a bug in the code when we look up our status, but there's not a bug. Take a look at the console, what do you see? Do you notice that the /orders request comes back before the /orderStatuses request? This is ultimately what produced the undefined result. Going back into the code, you can see that we try to look up the orderStatus description by comparing the current orderStatusId against a list of status IDs that we already fetched. But, if the orderStatus call returns after the orders call, then our list is empty, and this is what produces the undefined response in our application. Code like this is not unusual when people first learn JavaScript. Most people have written code like this, and they've seen it fail, just like this. We can fix this code so that the correct value returns, and displays in our results area. All we have to do is move a little bit of code around. We'll see what code we have to move, and where it has to go, next.

Building a Callback Pyramid

Dealing with APIs can lead to uncertainty. In the previous clip, we weren't sure we actually had the data we needed when we needed it. In this clip, we'll see a technique that we can use to ensure that we have all of the data we need before we use it. Returning to our demo page, we can perform roughly the same call, but this time we can ensure that we have our order statuses defined before we try to access them. Clicking the callback button will take roughly one and a half seconds, and then the data on the screen updates to show that our order status is in process. Looking again at the console, we can see that the order of our API results is reversed. With this button, the orderStatuses are returned before the orders data is returned. This ensures we have our data, but, how did we ensure that? Take a look at the code on the screen. The code itself is quite similar to the code you saw in the last clip. Once again, we have an xhr that is fetching the orderStatuses, and a second xhr that is fetching the orders. There's a slight difference though, did you see it? It's that our xhr2 is defined inside the xhr onload function. That is, we're waiting until the data of our first call came back before we even attempt to make our second call. By doing this, we ensured that we have all the data that we need for our second API request; however, it comes at a cost. Now our second call is buried inside the success of our first call. What problems could be caused by nesting functions in this manner? In the next clip, I'll tell you about two problems that you can run into if you nest your code this way.

Why Is the Pyramid Bad?

As we saw in the last clip, one way to solve the race condition problem is to use callbacks and functions inside of other functions. While solving this one problem, we've introduced another problem, the callback pyramid of doom. According to Wikipedia, a callback pyramid of doom is a common problem that arises when a program uses many levels of nested indentation to control access to a function. But why is it called the callback pyramid of doom? Well, let's start by looking at a simplified example. The code on the screen is example of code that could be called a pyramid of doom. It gets this name from the indentation on the left. If you tilt your head just right, it looks like a pyramid. This is the type of code that you might see in JavaScript when you need to make sure that A runs before B, and B runs before C, and C runs before D. But, what makes this a pyramid of doom and not just a pyramid? That is, why is this bad? The first reason is that it makes dirtier code, and exposes you to more errors. Let's revisit the code from earlier in the module with a slight tweak. This time, instead of just getting two pieces of data from the API, it's getting four, and each one is nested inside of the previous one. There's a lot going on here. Even though most of it's similar, you still need to process what's happening every time you read it, and in fact, there's actually a bug in this code. Did you see it? Inside the xhr3.onload function, instead of parsing the results that came back, you're parsing the results that came back on xhr2. It's a simple mistake to be sure, and it's one that when you see it in your code you'd likely chide yourself for being so careless, but it's also a common mistake. In fact, in preparation for this course, I did this exact thing in some of the sample code. Don't worry though, I fixed it so that you won't be seeing this error again in this course. Another problem with pyramiding code like this is how do you handle errors? If xhr3 returned

[00:06: 33.239]
a 500 from the server, then xhr4 wouldn't be called, and the one error function of xhr3 would get called, but it's inside the onload of xhr2, so, what would happen? If you said you should handle the onerror case, you're right, but, that can make your code even messier now. You now have a lot more code in your onload functions, and you're making the first problem even worse, but, beyond that, xhr2 and xhr1 calls were successful, so, their onerror functions aren't called, and now you're handling errors inside of a success function, which can lead to another series of messy decisions in how you write your code. So, how can we solve this problem without introducing a new problem? How can we write code so that it executes in order without massive nesting of functions? We'll learn how, next.

Solving the Callback Pyramid

As we've seen, nesting callbacks can lead to messy code. How can we get the

benefits of nesting without the messiness of nesting? The answer, as you might've guessed, is promises, but what is a promise? According to Mozilla, a promise is an object that represents the eventual completion (or failure) of an asynchronous operation, and its resulting value. Right off the bat, we see that it's an object that helps with an asynchronous operation, which is what we've been dealing with. It also tells us that it represents the eventual completion. That is, when a promise is called, it may or may not have a value. The value could come instantly, or it could come at some time in the future, like our orderStatuses call that takes one and a half seconds. Another way to think of it is that promises allow developers to write asynchronous code in a more clear and less error-prone manner, and that was a big part of my motivation when I first learned promises. Even though the concepts were not intuitive at first, I persisted, because I wanted my JavaScript code to be more readable. A promise can have three states. First, pending. This is the initial state, when you first create a promise, it's in the pending state. Since we're talking a lot about API calls, while the call is happening, the promise is pending. If you remember the orderStatuses call that took one and a half seconds to respond, a promise would've been in the pending state for the entire one and a half seconds. Second, fulfill. A promise moves from the pending to the fulfilled state when the asynchronous call has completed successfully. When a promise is fulfilled, it will return a single value. For example, a specific order from an API. The third and final state is rejected. A promise moves from the pending to the rejected state when the asynchronous call has failed. When a promise is rejected, it returns a reason why it was rejected; similar to the catch function in a try- catch block. There are a couple other terms that you might hear with promises. You might hear someone say that a promise is settled, or resolved. Both of these are referring to the same state. Whenever someone says a promise is settled or resolved, they mean that that promise is now in either the fulfilled or rejected state. That is, it means that a promise is no longer pending. One quick note while we're talking about the definition of promises; if you're coming from another language to JavaScript, you might already be familiar with a concept called promises. In some languages, these promises are a term used for lazy evaluation of code. In JavaScript, that is not the case. JavaScript promises are eagerly evaluated. Throughout the rest of the course, we'll see how we can get the values out of a promise, but it's important to note that promises do not wait for us to request the value before they execute. With that brief theory of promises, you're probably ready to start seeing them in practice. There's one quick step we need to do first, and I'll walk you through that, next.

Setting up the Sample Project

Are you starting to see the types of problems that promises are trying to address? Are you ready to get into some code and see what's actually going on? If so, great. Let's take a minute or two and make sure that we're all on the same

page with a sample project. For this course, we'll be accessing an API used by Carved Rock Fitness. They're an online sporting goods retailer. Their API allows them to create, track, and ship orders out to their customers. Their API consists of a few key objects; first of all, addresses, then products or items, next, orders, and of course, users. There's also a couple metadata entities; specifically there's ItemCategories, and OrderStatuses. This is but a small part of their API, but using these entities, we'll be able to check on the status of an order, or determine where the order's been shipped to. In order to run the API, we first need to get the code. Start by cloning the GitHub repo for this project, and while you're on that GitHub page, expand the list of branches. In addition to the master branch, there's four other branches. There's consuming-promises, creating-promises, iterating, and understanding-promises. Those four branches line up with the modules of this course. Each branch is the code that you need to see the functionality of the module. There's two ways that you can use this. First, you can start with the previous module's branch, and enter code as we go along. I hope this is the path you choose, because typing the code in will help to build some kind of muscle memory, so to speak, of how to solve some of the problems; and if you run into a problem that you can't get the demo to work, you always have that safety net of just checking out the correct branch when you're back up and running. The second option is to watch the course, and when it's time to run the demos, you check out the branch for the current module, and execute that code. The choice, of course, is yours. Now that you've cloned the code, go ahead and open it up in your IDE. I'll be using VS code, but any IDE you're familiar with will work. Make sure you're on the understanding- promises branch. There's a couple things to notice in this project. First, this project is built on top of JSON server. This gives us the ability to run an API locally without needing to set up a database. The data for this is located in the data/ db.json file. Since JSON server is an external dependency, you will need to run npm install before the project will work. Next, notice that there's a series of .html files. Specifically there's consuming, creating, index, iterating, and understanding. The index.html file is the root of the website, and as you might guess by now, the other files are tied to specific modules. You will not need to edit any of the HTML files throughout this course. They're set up and ready for you to run. After all, this is a course on asynchronous JavaScript programming, not HTML coding. That leads us to the last thing to note. In the source folder, there are five mjs files. There's one for each of the modules, and a fifth one named results.mjs. This file is a helper file that will display some information on the screen for us. As with the HTML files, you will not need to modify this file during the course. Open up the understanding.mjs file, here you can see the code that you've already seen in this module. If you open up consuming.mjs, which is the next module, you can see five empty functions. You'll be coding the body of these functions as we go throughout the course, speaking of which, are you ready to start coding and seeing how promises can help you in your JavaScript coding? We'll jump right in, in the next module.

Consuming Promises

Consuming Promises

Now that you've seen a glimpse of why you'd want to use promises, it's time to start seeing promises in action. For most JavaScript developers, the first time they'll encounter promises is when they consume another library. An example of this is one of the most popular HTTP request libraries, Axios. At the time of this recording, it averages six million downloads a week. This library is an abstraction on top of the XMLHttpRequest calls that you saw in the last module. One of the key abstractions is that it is promise-based; that is, you can't use Axios without using promises. To see this in action, inside of your IDE, open the consuming.mjs file. Inside of the Git function, add the following line. A quick side note, the Axios library is already included in this project, and referenced in the HTML files, and that's why you have access here without doing a npm install. Next, run the command, npm run dev. This will launch the web server so that we can see our code. If you go to localhost:3000, which is the root of the web server, you won't see our landing page, instead you'll see this API landing page for our JSON server API. However, if you go to /home, you will see the landing page for the course. Click on the Consuming promises link, and that will take you to this page. We'll spend all of our time in this module on this page. Before continuing on, make sure that you have your console open. Now, run your function by clicking the Success GET button. Nothing on the website changed, but if you have XHR logging turned on, you'll see in the console that a request was made. If you don't have that turned on for the console, you can jump over to the network tab, and verify that that same request was made. This shows us that Axios did indeed make a request for us. I want to take a quick sidebar here. This is an example of what I meant earlier when I said that promises were eager, not lazy. We've not yet done anything to actually get the data from our promise, and yet it still kicked off a request to the API. So jumping back, what do we need to do to update our results in the box? Head back into your code. Earlier, we talked about how a promise can have three states; pending, fulfilled, or rejected. We want to handle the fulfilled state, that is, we want to make sure that when the request succeeds, we can access the data. To do that, we use a function named then. So add then to the end of your Git command. When a promise is fulfilled, it will call then functions. Then takes a function as its only parameter, and this function is executed whenever the promise succeeds. So, let's add that function now. Before we execute this, let's do a very brief walkthrough. Inside the function passed into then, we're destructuring the result that is passed back. In this case, Axios returns a result that has a lot of information about the request, such as headers. It places the payload in a property named data, that is, the data property contains our order that we just requested. Next, we're using the setText function, which is in the results.mjs file I mentioned in the last module. This

function will take whatever data we give it, and place it in our results box on the screen. In our case, we're just going to pass it a string version of the data that Axios retrieved for us. Switching back to the browser, reload the consuming promises page to make sure it has your up to date changes, and this time when you click on the Success GET button, your response box will update with the details of the order for order number one, and just like that, you've consumed your first promise. Let's look at the code quickly one more time before we continue on. This function is the essence of a promise. It starts by calling an asynchronous function. In this case, it's an HTTP get request. Next, it chains a then function onto that request. Essentially it's saying, after the get completes, run this function next. Remember, part of the definition of a promise is that it handles the eventual completion of an asynchronous call. We're not concerned about when the then call happens, but we know it happens after the get call succeeds. That works great when everything works, but what happens when your API request fails? You'll see how to handle errors, next.

Handling Errors with Promises

There are countless ways for any asynchronous function to error out. Even in our case with HTTP requests, you could get a 404, or a server error, or a handful of other status codes that indicate that the call was unsuccessful. Let's see how to handle those. Return to your IDE in the consuming.mjs file. This time, we'll be adding code to the getCatch function, but let's start by copying code from the get function above. We can use this as our starting point. After you paste it in, make a couple small tweaks. As I mentioned, the Axios library returns a lot of data as part of the result object. So let's change our function to pass in a result object, and not just data. Next, let's examine the result.status code, and this is the HTTP status code of the request. So let's add an if condition around our setText. This way, we can check our condition, and set the text to an error if it was not a successful request. Now, let's force our request to be unsuccessful by requesting an order that we know does not exist. So change it from orders/1 to orders/123. With all that done, return to the browser, and reload the consuming page. Click on the Failed GET request. You'll notice that the results box didn't update, it still just says results, and in your console, you now have two errors. First, you have one for GET http://localhost:3000/orders/123 is a 404, and the second one is that there is an uncaught error, specifically in the promise; and you can also see that the XHR request failed. The first error is expected. We're trying to get a record that we knew didn't exist, so we expected a 404 to come back, but the second error is unexpected. What does Uncaught (in promise) mean? Returning to your IDE, recall the three states of a promise; pending, fulfilled, and rejected. The last clip showed how to handle a fulfilled promise, but this time, our promise wasn't fulfilled. It came back with an error, which means that we're in the rejected state, and the rejected state is not handled with the then function. So, let's reset our code back to what it was at the

beginning, and then change that ID back to 123 to make sure we get an error. What we need to do is call a different function to handle the rejected state. That function is named catch. So chain a .catch after the then function. Much like the then function, the catch function also takes an internal function. The internal function only needs one parameter, and that represents an error or reason. Whenever a promise is rejected, it will pass in a reason for why it was rejected, and all we want to do here is display that error, so we're going to use the same setText function to display our error. Refresh the consuming page, and one more time, click on that Failed GET button. Now the results box shows the error that the request failed with a 404. Additionally, when you look at your console, you should still see an error that the get request returned a 404, and you should still see a message that the XHR request failed, but you should not see an Uncaught (in promise) error, because you've now handled the error case of your promise. Briefly return to your code. Notice that inside the catch, all we're doing is displaying the error. That's pretty much solely for demonstration purposes. If this was a real application, you might take other actions here. What you do inside of your catch block is dependent upon the use case of your particular application. Additionally, what data the error or reason contains is up to the library that you're consuming. You've now seen a pretty simplistic usage of promises, but what about that case from earlier in the course where you needed to order your requests? Up next, I'll show you how you can handle that exact situation with promises.

Chaining Promises

So far, you've seen the simple case of making a single asynchronous call, and handling its success or failure, but, let's be honest, you probably didn't need promises for that, you've been doing that on your own for a while. So, how can promises help ensure that the right data is loaded when it's needed? One nice thing about promises is that the settled functions, the then and catch, both return promises. This means that you can chain your promises together. For example, if we were to look at our success code from earlier in the module, we could have our then return another value; nothing special, any ordinary return value will work, and the then function will then wrap this return value in a promise, which means that we can chain another then to that function. Now when this code runs, it'll place the order on the screen, and then it would log out the word, Pluralsight, to the console. We've leverage this ability to be able to sequence our data requests. Staying in that familiar consuming file will modify the chain function for this clip. As we did in the last clip, let's copy the body of the get function to get started. This time, instead of display8ing the order on the screen, we want to know what city is the order being shipped to? In order to do that, we need to first load the order, and then look up the shipping address in a second API call. Start by modifying the internal function of then, delete the setText function call, and instead make a second axios.get call. Then, let's handle

that call. We could add a .then where we're at now at the end of our second get, but now we're getting right back into that problem of the callback pyramid that we discussed before. So instead of chaining the then off of the second get, let's chain it off of the other then. With that code written, let's reload the consuming page in our browser. Before clicking on the promise Chain button, think for a second about what you'll see. When you clicked the button, did you see your expected results? Were you expecting to see something like CITY : OMAHA in the results? Instead, it still just says results, and down in the console, there's an error. So, what's going on here? Why didn't it update the results? Remember that we just talked about how every then and catch returns promises, and how that allows us to chain them? Well that still happened here, but our first then we didn't actually return anything, and what happens in JavaScript when you don't specify a return value? It returns undefined, and that undefined value gets wrapped into a promise, and passed onto the second then, and then your code is trying to destructure data from an undefined, and it fails. So, how do we solve this? The answer is really simple, and it's another one of those things that I've forgotten to do more times than I'd like to admit. We have to make sure to return the axios.get call, so that we're returning the correct promise. After you add the return statement and refresh the page, click on the promise Chain one more time. This time, the results show up in the box to say CITY : BOWLING GREEN. That is, order number one is going to ship out to Bowling Green, Kentucky. Look at the code one more time. Let's read this code from top to bottom. It's going to start by getting an order, then, it's going to take that order and read an address, and then it's going to display the address. Our code is starting to become more clear and readable by using promises, and you can imagine how you could chain several promises together in this fashion, as long as you remember to return the promise and each then. But if you're like me and you forget sometimes, or more than sometimes if we're honest, I'll show you how you can handle that situation more gracefully, in the next clip.

Catching Errors in a Chain

Sometimes when you're chaining promises together, the unexpected can happen, and as you saw in the last clip, sometimes that can blow up. We saw that happen when it logged out the exception to the console. So, how can you make that a little bit smoother? Staying inside the consume file in your IDE, this time we'll be writing code in the chainCatch function. Start by copying the code from the chain function that you just wrote. But in order to get the error that we got in the last clip, make sure to delete this return in the first then function. This will ensure that we get the same destructuring error that we got last time. Next, we want to add a catch function. The question is, where do we want to add it? Well let's start by placing it after the second then. Return to the browser, and reload the page, and make sure to click on the Chain Catch button. When you do, you'll see the results block update with an error that it can't destructure data from an

undefined or null value. This is the same error that we saw in our console last time, but as you look down in the console now, it's not there. That's because we've caught it in this promise chain, and handled it by displaying the error to the user. Returning to our code, let's take a look at another way that we can use this catch function. Start by adding back that return statement on our address, so that it completes successfully. This time, inside of the second then, introduce an error by adding a .my after the data. That is, we're trying to get the city from the myObject. Save this code, go back to the browser, and reload. Once again, click that Chain Catch button, again, there's no console error this time, but the results block now shows cannot read property 'city' of undefined. The catch that you added will catch any error in your stack. Looking at the code once more, you can see that the catch is the last statement in the chain. It'll catch any error in any of the previous calls. But what if you wanted more fine-grain control? Well, you can actually have that with catch. Instead of returning the data from the address call, have it throw an error. Then, add a catch right after that first then. Notice that that first catch still returns data, and in fact, it's returning an object with a data parameter. If it did not return that, then the second catch block would also be hit when the second then block tried to destructure data. In this case, the first catch will only catch errors from the first then; however, the second catch will catch all of the errors from the first catch through the second then. To see this at work, change the return in the first catch to throw a new error. Then, reload the consuming page, and click the chain button one more time. The result block shows the text of second error, which is that second catch block that you just wrote. Going back to the code one last time, let's put everything back to the way it was. While you do have the power to catch at multiple locations, most of the time you'll probably just want one catch statement. If you do decide to add multiple catch statements in your chain, remember to make sure that you're handling the error thoroughly; otherwise, if you just return undefined from your catch, your code will run into additional errors. As you start to chain promises together, you might want some code to run only one time after the entire chain is settled. You could have a lot of duplicated code in each then and catch block to ensure that you're capturing that the promise is completely settled, or as I'll show you next, you could accomplish that with a built-in promise function.

Performing One Last Operation

We've already seen how to handle a promise getting fulfilled, and what to do when a promise gets rejected, but what if you need to execute some code after a promise settles, but you don't care if it was successful or not? Thankfully, there's a function just for that. As we've already seen, one common case for consuming promises is using a library like Axios to make HTTP requests, and often, when applications are making HTTP requests, they like to show a loading indicator, and then once the call is fulfilled, they would want to hide the loading indicator, but we also want to make sure in those situations that the indicator is hidden when

the call is rejected. I think we've all ran across a website that doesn't hide the indicator when there's an error, and you're unable to do anything. So we need to make sure that the indicator goes away whenever the promise is settled. However, we don't want to do it the instant a promise is resolved. We want to make sure all the code in our then and catch blocks completes before we hide the indicator, particularly, if we have a long-running process in one of our then blocks. We don't want the indicator to disappear too quickly, so, how do we handle this? Return to the consuming.mjs file, and by now you know the routine, we have one empty function left, so we'll be placing our code in that block in this clip. Start by pasting the code from the chainCatch function into the function named final. Next, we want to show our loading indicator, and that's imported from our results file. So before you make the Axios request, call showWaiting. Let's hop over to the browser, and make sure that we know that the loading indicator is working before we continue. Refresh the app, and click the final button. When you do, a green box covers the screen, and it says, waiting. So our loading indicator shows up, but unfortunately it's not being hidden. Going back into the code, we have a couple choices. We could place our hideWaiting function in the then block, or the catch block, or both. We already said that we need to make sure both states stop showing the waiting indicator, and we started our course by talking about producing good code, so, it's probably best if we don't put it in each place repeating our code. Thankfully, promises have a function just for this case. After the catch block, add a new function named finally. Inside this function, let's use a timeout to call our hideWaiting. We normally wouldn't want to set a timeout for this, but for demonstration purposes, it'll let us see the waiting indicator for at least one and a half seconds before it disappears, and then we're going to update our result to let us know that we're completely done. Return one more time to the browser, and reload, and then click the Final Update button. You can see the waiting blocks show up; then after about one and a half seconds, the box goes away, and we see that our text has a –completely done added to it. That's because our finally block waited until all our code was done, and then ran. This allows you to run some asynchronous code, handle the success or failure case, and then run some final code. In our case, we're using that function to clean up our waiting indicator. Remember, there's three states to a promise. There's pending, fulfilled, and rejected; and in this module, you were able to see each of these three states, and more than that, you were able to see how to start chaining fulfilled and rejected states to handle the flow of data in your application; and that all works great if you're using a library like Axios that's already built on top of promises, but what if you want to work with an async function, like that setTimeout that we just used in our finally block? Is there a way that you can turn that function into a promise? There is. In the next module, I'll show you exactly what you need to do to create your own promises, starting with set timeout.

Creating and Queuing Promises

Creating Promises

By now, you've seen some of the power of promises. As your code transitioned from being nested function inside of nested function, to chains of thens and catches. To truly get the power of promises though, you need to be able to create your own promises, and not just use them from other libraries. As you think through consuming promises, and you start to think about what you'll need to do to create your own, what are the major aspects you need to consider? You need to consider the three states and how you'll manage them; pending, fulfilled, and rejected. Pending is probably the easiest state to handle, because remember, a pending promise is just a promise that's not yet settled. When you create a promise, it's pending until you tell it to move to the fulfilled state or the rejected state. This leads to the question of how do you create a promise? Before you answer that, let's look again at the definition we saw earlier in the course. A promise is an object that represents the eventual completion (or failure) of an asynchronous operation, and its resulting value. We've already talked about eventual completion, and we've talked about asynchronous operation, but do you see anything in the definition that tells us how to create it. Well it starts with the first word, a promise is an object, and this is an example of how you could create one, just like you would create any other object in JavaScript. At this point, temp is a pending promise; so, now that you know that, how do you make it change states? Since we're in the creating promises module, let's open the creating.mjs file, just like we did in the past, and we'll start with our first function, in this case, it's timeout. We want to take the asynchronous function, setTimeout, that we saw at the end of the last module, and turn it into a promise, so we can take advantage of all of the benefits of promises. Let's take the code we just saw in creating a new promise, and use it here, but this time, let's call our variable, wait. Our promise is now in the pending state. We're eventually going to have it switched to the fulfilled state, so, let's handle that situation now. Now that we have that out of the way, we need to figure out how to make a promise change states. The first thing to know is that a promise takes a function as the one and only parameter to its constructor. This function is called the executor function. So, update the line that creates a promise, and pass in an empty function as the executor. Next, let's put our setTimeout inside that executor function. Since, as we've seen, promises are eager, this code will start to be executed immediately, which means our empty function will be called 1500ms after we new up the promise. But the function running doesn't change the state. The executor function does take a parameter, resolve. This parameter is a function that the promise can use to call and resolve its state, so let's add that in now. Now we need to call our resolve state when we're ready for the promise to be fulfilled. So let's add resolve to the anonymous function inside of setTimeout, and let's pass the string, Timeout, to the resolved function. This will be our return value. We now have a promise that will call resolve, which will set the state to fulfilled, and then call our then function. Let's see all of this in

action. If your server's not running, make sure you run the command, npm run dev, before you head over to your browser. Once you're in your browser, navigate to localhost:3000/home, and then choose the Creating Promises link. Click on the Timeout link. After one and a half seconds, the result box updates to show our timeout text. Our then function got called one time, and only one time, but then again, the setTimeout function will only fire once, after 1500ms. In the next clip, you'll see a key illustration that will help demonstrate promises states using a function that will update multiple times.

Understanding Promise States

What happens when a function that's been promisified fires multiple times? Understanding this will help you understand promise states. Let's go back to our creating.mjs file; start by copying code from the timeout function into the interval function. Next, replace the setTimeout function with setInterval; setTimeout and setInterval are very similar. The main difference is that setTimeout will only call its internal function one time when the timer expires, but setInterval will call multiple times, once every time the timer expires. In our case, that's once every one and a half seconds. To see this in action, let's create a counter outside of setInterval, and update it inside of our function. Then, let's add a console statement so that we can see every time this function fires. After we write that code, let's jump back to the browser. As always, reload the web page to bring in the new code, and then click on the interval button. After the first one and a half seconds, you'll see the response box update to TIMEOUT! 1, and INTERVAL logged out to the console. After the second one a half seconds, the result box still says TIMEOUT! 1, but a second interval is logged out to the console. In fact, no matter how long we sit and wait, our results never update past TIMEOUT! 1, but at the same time, you can see that the interval count keeps rising, so we know our internal function has been called. So, what's going on here? To answer that, hop back into the code. Let's add a finally block at the end of our then statement. Before we return to the web, what do you think the counter will be in finally? Once you have your answer, go back to the browser, and refresh the page, and then once again, click on the interval button. Once again the result box shows TIMEOUT! 1, this time though, it appended that DONE 1. So does showing DONE 1 shed any light into what's going on? Remember earlier in the course when we talked about states? I mentioned that there are two other words that are used to reference when a promise is no longer pending. Those words are settled or resolved. Based on the last demo, you can see that once a promise is settled, its state is not updated. That's because if the associated promise has already been resolved, either to a value, a rejection, or another promise; this method, the resolve method, does nothing. That is, once the promise is settled, it's done, and attempting to settle it again will have no effect. It's also why the finally function in our last demo had the counter as one. The function itself continues to run, but the promise will not change. So, how

can we make the interval stop? Go back into your code, and copy the interval code into the clearIntervalChain function. After we do that, we need to make a couple tweaks. First, before the promise is created, create a variable named interval. Next, assign the return value of setInterval to the new interval value. Lastly, inside that finally block, call clearInterval, and pass in your new interval value. This will stop the interval. Returning to the web and reloading the page, you can click the Clear Interval button. As before, you can see the TIMEOUT! 1 in the results box after about a second and a half. You can also see the interval logged out to the console, but if you wait there, you won't see another interval logged out, because in our finally block, we cleared the interval. It's another example of using the finally function to clean up after a promise. If moving a promise to the fulfilled state is that easy, then surely moving it to the rejected state is just as easy, right? Well we'll find out in our next clip.

Rejecting a Promise

The previous two examples were overly simplified. It's not that resolving is more difficult than what you saw, but those two promises don't have a way to set the promise to the rejected state. You'll see how to do that, now. Let's head back to the creating.mjs file, and go to the xhr function. In this function, we're going to make an XHR call, similar to what we did earlier in the course, but this time we're going to put it inside of a promise. As we saw earlier, XHRs have an onload function for successful calls, and since it's the success call, that's where we want to fulfill our promise. So, let's add our resolve to the promise executor function, and then call resolve for onload. At this point, we're right back to where we were with the previous examples. We created a promise, and then set it to the fulfilled state when something happened. Next, we need to handle the case when something goes wrong. In our case it's going to be a 404, because user ID 7 doesn't exist in the Carved Rock Fitness API. We know that there is an onerror function, but how can we use the onerror function to change the state? What if I told you that the executor function took a second parameter? That second parameter is the reject function, so let's add that. Now all we need to do is provide a then and a catch function for our request promise. With that, hop on over to the browser. Reload the page, and then click on the XHR button. The response shows an empty object, but it doesn't say request failed, so that means our promise wasn't rejected. Go back to the code and look at the XHR code. The problem is not with the reject function. The problem is that XHR only calls onerror when there's something like a network error, and the request failed to be completed. All other calls wind up in the onload function. So, let's tweak that to check for XHR status. Now return to the browser, and reload. Once again, click the XHR button. This time the result shows up as NOT FOUND, because that was the response from the server. This shows that you have the power to control when your promise is fulfilled or rejected. The code we just wrote had multiple reject statements. That's because there were multiple reasons why the call could

fail. You could also have multiple resolve calls if they're in different paths. You've already seen how to call a single promise, or even how to call a second promise after the first promise has settled. In the next clip, I'll show you one function that will let you queue up several promises at once, and then wait for them all to be fulfilled before continuing on.

Waiting for All Promises to Resolve

Have you ever needed several functions to run independent of each other, but you couldn't continue until they were all complete? If so, that's a use case that promises excel at. One of the first large-scale applications I wrote using JavaScript required a lot of metadata. There were users, and customers, and jobs, and accounts; and each of those had their own statuses and types, and other pieces of metadata. We might've needed 10 pieces of metadata to perform one single action, but I didn't need to load the user metadata and then load account metadata, because the data wasn't really related, I just needed to make sure that all of the metadata I did need was loaded before I continued. In fact, not only did I not need to have sequential API calls, I didn't even want them, that would've been too slow. I wanted to batch up all of my calls, fire them off all at once, and then move on once they all came back. This is part of the power of asynchronous programming. You don't have to wait for one call to finish before you can start another, and with promises, you can still tell the code to not continue until all of the data comes back. Our code in this clip is going to load all of the metadata from the API, specifically, that's item categories, order statuses, and user types. Return to the IDE, and the creating file once again. Navigate down to the allPromises function, as this is where we'll be spending our time in this clip. Since we're loading data from an API, let's use Axios again. Now remember, Axios is built using promises, so each of our variables here is a promise. The question is, how can we wait until all three of these promises are fulfilled, particularly when we don't know the order that the API will return them in, and in reality, that order might change from call to call, depending on the API's load, and other considerations. Since we want all of the promises to be fulfilled, we should use the all function on a promise. To do this, we do not need to create a new promise object, instead we call Promise.all, and then we pass in the three variables that we just created. This will queue up all three promises, and wait until all three return. We can attach a .then function to this all function, so that we can handle it in the same way as before, but what will be the result value in the then function, do you have any ideas? It's going to be an array of results, and the order will match the order that we added them, not the order that they were resolved, and that's good news because it means categories will always be the first result. Remember that these are Axios objects coming back. Therefore, we need to access the data property on them to see our data. Hopping over to the web, if we reload our page and click on the All button, after about one and a half seconds, we see a list of data dumped out. As

we scroll, we can see that that includes item categories like kayak, and order statuses like shipped, and user types like individual; and if you look at the console, you can see that all three calls were made to the API, and even though the first two responded nearly instantly, the call waited until the order status call completed. Remember, this call has a one and a half second delay in the API. What happens though if one of our calls fails? Will it sit there waiting endlessly for it be fulfilled? Well, let's tweak our code and find out. Start by adding a fourth Axios call, then add it to the promise array, and of course, add it to the then as well. With that, hop back to the browser, refresh, and rerun the all command again. This time, nothing is updated, and we have an error in our console. Do you remember what we have to do when we have an Uncaught (in promise) error? That's right, we've got to catch it. So, add a catch statement after the then. Save this, and head back to the browser. Reload, and rerun that all function one more time. Notice what happens; not only does the box update nearly instantly with a 404 error, but if you noticed in the console window, it updated before the order status call even completed. Let's rerun that, and pay attention to the console. The all function will wait until either all promises are fulfilled, or until the first promise is rejected. This can be useful in a situation where you don't want to continue if any of your promises are rejected. For example, if the address types data had been something so essential that we couldn't possibly continue without it, we wouldn't want to wait for other promises to complete before continuing on. But what if that's not the situation? What if the data is so independent that you don't really care if one or two calls failed, and you still want to wait until they're all settled? I'll show you the two things you need to change to make that happen, next.

Settling All Promises

Sometimes when you queue up promises, you want as much data as possible to come back. If one or maybe even two promises fail, that's okay, so long as you get the rest of the data. To see how to do this, open up that creating.mjs file, and modify the code for the allSettled function. Start by copying the code from the previous clip, the all function, into this function. The first thing to change is the name, instead of all, it's allSettled; allSettled is similar to all, but it's different enough that I want to pop out to some slides to explain it before we use it. First, the data that's passed back is different. All returns the results object as part of an array, but allSettled returns a different shape. It has two keys. The status key will be either fulfilled or rejected, and then the second key will be either value, if the status is fulfilled; or reason, if the status is rejected. That is, the then function on allSettled will return all promises even if they were rejected, which leads to a second difference. We don't need a catch block, because this promise will resolve with an array of data, including rejected promises. Even though a catch block is not specifically needed, it's still a good practice to include. It will help catch any errors that might occur inside of your then block, for example. With that very brief

explanation, let's hop back to the code and take a look at what this would look like. Start by changing what's received in the then function. Instead of individual values, simply pass in a values parameter. Next, let's add some code to show what the status is, and a glimpse of the data we get back, and to do that, we can delete all of the body of the then function. This code first of all checks the status, and it checks to see if it's fulfilled. If it is fulfilled, then it constructs a string that includes the data that was passed back, and in this case, it's only taking the first element of that array. If it was not fulfilled, it'll set the status to REJECTED, and give the reason why; and notice with both of these strings, there's an empty space at the end, and that'll just help for formatting on a smaller screen. Reload the website in your browser, and click the All Settled button. The results on the window appear after one and a half seconds, even though the addressTypes call rejected nearly instantly. The data shows the first three promises were all fulfilled, and the last one was rejected with a 404. By changing the name of the function from all to allSettled, and by expecting a different object shape in the then function, you're able to queue up several promises that wait until they're all settled before calling the then function. One thing to note here, as of the time of this recording, not all browsers support allSettled, the ones that do are primarily Chrome and Firefox. To check the status, check out the MDN page listed on the screen. In the next clip, I'll show you how you can use promises to get data from the fastest API endpoint, and ignore all the other data.

Racing Promises

Earlier in the course, we saw how race conditions can cause problems, but with promises, we can actually use them for our advantage. Imagine that you've got a few copies of your endpoint deployed, perhaps in some geographically diverse locations. The time to get data from these various endpoints can vary based on where the request is coming from. What you'd really like is to get the data from the endpoint that is fastest for you. You don't care which endpoint that is, so long as it's fast. Promises can help simplify this. Your default instance of the API is already running on localhost 3000, but from your terminal, run the command, npm run secondary. This time you see that the server is now running on localhost 3001. In fact, by running both the dev and secondary commands, you have two instances of your API running. Let's use these instances to see how promises can help with the data. Inside your creating.mjs file, jump into the function named race. Start by creating two requests; the first one will get the list of users. The second request is also to get the list of users, but this time it's a different endpoint, it's localhost 3001. The next step is to fire off both of these requests, but we want the results of the first one to settle, so we won't use all or allSettled, because those would wait for both of them to settle. Instead, we'll use Promise.race. Next, we need to add a then function. This will return with the first promise to settle, so we'll only be a single value. Finally, we need to add a catch handler. As long as you have both the dev and secondary endpoints

running, head over to your browser, and reload the page. Click on the race button. When you click this, the data almost immediately comes back, and it shows up in the results pane; and if you look at the console, you can see both requests were fulfilled, and that in my case, the localhost 3000 request was faster. But the point is, for this case, it doesn't matter which one came back faster, since they're duplicate endpoints. One important note here; race will return whenever the first promise settles. If it settles with a rejection, that will call the catch function, and you won't get your data. Since race will take the response of the fastest promise, it's not going to have a lot of use cases. Most of the time when I'm queuing up promises, I will use all, because I want to use the data once all my promises are fulfilled, but you should still be aware of the race function even if it's rarely used. Earlier, you learned about consuming promises. In this module, you learned how you could create your own promises, and hopefully you learned that creating them isn't too scary. You create a new promise with a basic instructor. Then you pass in an executor function with a resolve and reject parameter. You can use these parameters to set the state of your promise to either fulfilled or rejected, and you're in complete control of when each state should be set. You even learn that there's three distinct ways to queue up a list of promises; all will wait until either all promises are fulfilled, or until one promise is rejected; allSettled will wait until all promises are settled, either fulfilled or rejected, but it's not supported in every browser; race will only wait until the fastest promise is settled. With these functions, you have a lot of control over how promises are used in your application. A few years after promises were developed in JavaScript, the language came up with async/await, and I think now that you know the basics of promises, you're ready to learn some secrets about async/await. The next module will help demystify this operator, and once you understand one important fact about async/await, your knowledge of promises will transfer to this new syntax as well.

Iterating with Async/Await

Iterating with Async/Await

Now that you're familiar with promises, it's time to reveal an uneasy truth; JavaScript Promises are kind of old. For some people, they've already been replaced by async/await, but don't worry, what you already know about promises will prove to be a solid foundation for async/await. If JavaScript already had promises, why did it introduce async/await, or in other words, what's the point of having an async/await in JavaScript. The purpose of async/await is to make it easier to use promises in a synchronous format. In other words, don't think of async/await as a new technology or paradigm, instead, think of it as syntactic sugar for promises. Syntactic sugar is just syntax within a programming language that's designed to make things easier to read or to express, so the goal or

purpose of async/await is to make working with promises easier. It simplifies some of the code, and makes it more clear exactly what's going on. All of this is good news, since you've already learned a lot about asynchronous programming in promises, your understanding of async programming will carry forward to async/await. So while you're going to learn more about asynchronous programming, you'll be starting with a solid base. Before getting into the examples, let's take a minute or two to define what async/await actually is. To start, there's two separate keywords; the first one is async, and as you probably guessed, the second one is await. The asynchronous keyword is used to designate that a function is asynchronous. This keyword is used when functions are defined, so you can use it with either the traditional function declaration like you see on the left, or with fat.arrow function declaration like you see on the right. In either case, this function will return an implicit promise, which should be somewhat comforting, because underneath the function, it's still just a promise that's being operated on. Because it's an implicit promise, it means that whatever you return will be wrapped inside of a promise. Additionally, if your function throws an error, that will be wrapped in a rejected promise. The awake keyword pauses the execution of an asynchronous function while it waits for the promise to be fulfilled. There's a couple of important things to note about await. First, it can only be used inside of an async function. If you try to use it outside of an asynchronous function, you will receive an error. Second, it only blocks the current function; however, it does not block the calling functions. For example, if you had the code on the screen, the await for someFunc would halt getNames and not go to the doSomethingElse function, that is until someFunc was done. However, it would not stop getAddresses from executing. Don't get too hung up on the details though, as the rest of the module will get into some code examples for you to see how it works. The important thing to note is that both async/await and synchronous promises are both trying to accomplish the same thing. The next clip will show you how you can take an asynchronous HTTP call that you've already done in the course, and turn it into a single line of code.

Awaiting a Call

Now that you have a brief overview of async/await, the only thing left to do is dive into some examples. Start by opening the iterating.mjs file. We're going to write code in the get function. Much like we did with consuming promises, we want to do a simple HTTP request. The code here has the await keyword before the axios.get function. This highlights that you can use await on functions that return promises. That is, you don't need a separate async/await version of Axios. In fact, that doesn't even exist. Additionally, since this is now making the promise a synchronous call, you can assign the data parameter to the value of the get call, or more accurately, you can destructure the result of the get call into a data parameter. Before you load up the web browser, make sure you're running npm

run dev so that this code will be executed. Go into the browser, and choose the iterating link to load the functions for this module. When you load this page, you actually get an error in your console. It tells you there is an unexpected reserved word. If you click on the file name over on the right, it takes you to the code that you just wrote. Can you see where the error is? Well let's go back to the code and look at that get function. Does looking at the code in your IDE help at all? Well remember the last clip? I said that there was two important notes on the await keyword. The one that applies here is that it must be used inside of an async function. So add the async keyword to the function definition. Save that, and return to the browser. This time when you reload the page, you don't see any console errors, or at least you shouldn't. So now you can click the get button. You once again see the familiar output in the results pane. Let's take a quick look at the code side by side. On the left, we have our promise-based approach from earlier in the course, and on the right, we have our async/await- based approach. They're both accomplishing the same thing, and they're doing so in similar ways. However, with the await, it might be slightly more clear the order that the code is getting executed in, but also note that they're really not that much different. Much like with promises, what happens here when a call like this fails? The next clip will show you a familiar way to handle errors with async/await.

Handling Errors with Async/Await

When handling errors with promises, you only had to attach a catch function, but how do you handle errors with async/await? Head back to the code and copy the get function code into the getCatch function, and make sure that the getCatch function is async, so you don't get that same reserved word error that we got in the last clip. As we did with our promise code, change the route to getOrder123, because that'll produce an error for us. In the last clip, I promised you that it's a familiar way to handle the error. You want to take a guess at maybe what that way is? Well, believe it or not, this is one place where it's not really related to promises, that is, there's not an awake catch function. Instead, it's the tried and true try-catch block. If you reload the website and click the Handle Errors button, you'll see a familiar 404 error in the response text, and you'll notice the standard 404 error in the console as well. That shows that we're handling our promise rejection. Looking at this code one last time, notice that since the error handling is the standard JavaScript try-catch block, you now have the ability to have the same error handling for both your synchronous and asynchronous code if you're using async/await. Earlier we saw how promises can be chained to combat race conditions. In the next clip, I'll show you how to accomplish the same thing with async/await.

Chaining Async/Await

Async/await makes sequencing asynchronous calls even easier than promises did. Start in the IDE, but this time go into the consuming.js file, and take a look at the chained function to be reminded of what we did with promises. This function loaded a specific order, order number one. After we got back our data from the API call, we made a second API call, this time to get the shipping address for our order. Finally, we used the data that was returned from the address endpoint to get the city where our package was shipped. We want to do the same thing, but this time with async/await. So, switch back to the iterating file, and notice that this file also has a chained function. That's where we'll be adding code for this clip. As with other functions, let's make sure that this function is async so that we can use await inside the body of the function call. Next, let's add our first API call. Now that we have our data, we want to get our address data, so let's add that API call. This time, we're destructuring data, but we're assigning it to a variable named address. Finally, we'll output the city. Heading back to the browser, we can refresh the browser, and then click on the chain button. When we do, we get the results pane updated with CITY: BOWLING GREEN, which is what we'd expect. It's the same result we got with promises. Also, looking at the console, we see the orders call is the first call, and the addresses call is second. This is also expected, since we sequenced our API calls in that order. Let's compare our code for accomplishing this fete that we've seen throughout the course. We'll start with our original async code. We had XHR functions that called other XHR functions inside of their onload function. We upgraded this to use promises, and then we were able to put our next API call inside of a then function on a promise, and the promise code improved the XHR code because it allowed us to chain multiple then statements together to handle the flow, as compared to nesting multiple calls, and finally, compare that with the async/await code. This appears to be an even better improvement than from XHR to promises. The code is a bit more straightforward, and if you wanted to chain another HTTP request, all you'd have to do is add another await statement with the next API call. The challenge is that handling async/await in this fashion makes all of the calls sequential. So what if you want to make concurrent calls? I'll show you how async/await can break free from the sequential calls, in the next clip.

Awaiting Concurrent Requests

We've been using async/await in a way that makes sure to call asynchronous functions in a truly sequential fashion, and sometimes, that's exactly what you want, but other times you don't want the functions to depend on other functions, and don't worry though, you can still use async/await for that situation. Go back into your iterating file. This time make the concurrent function async, and next, let's add a call to our orderStatuses API. Remember, this endpoint takes one and a half seconds. Notice that this function is not awaited, instead, the return value, a promise, is assigned to a variable named orderStatus. Next, let's add a second function call, this time to get the

orders. Again, this function is not awaited. Next, let's set the text to be an empty string. This lets us clear the text so that we can see the data when it comes back. Keep in mind that axios.get is still a promise, and promises are eager, as we talked about at the beginning of the course. That means that the request has already been kicked off. Next, let's await our two functions, and then after those two statements are awaited, we append our data to our response text so that we can see both results. Heading back to the browser, make sure to reload the page, and then click on the concurrent button. After a brief pause, you'll see the data all appear at once. Before returning to the code, take a look at the console and see that the orders API call resolved first, and that the orderStatuses call resolved second. By not placing an await on our Axios functions, we allowed both API calls to kick off at one time. Returning to the code, take a look at the await statements. We're awaiting the orderStatus first, which means we won't do anything with our orders data until after we get the data for our orderStatuses data. But because orderStatuses is slower, by the time the await is complete, the request for orders is already complete, so both calls were running at the same time, even if we did tell the code to wait on the slower request first. There's one more possibility for these calls. What if we wanted to handle the same two calls, but show the data in the order that it comes back? In the next clip, you'll see how to handle that exact case.

Awaiting Parallel Calls

Part of the power of asynchronous programming is that you can make parallel calls so that you're not blocking a fast- running process with a slow-running process. You can do that exact thing with async/await. Start as we've done every time in this module, by making the parallel function async, so that we can use await inside of it; next, let's set the text to be an empty string like we just did with our concurrent calls. Before we add more code, remember that async/await is syntactic sugar on top of promises, so, we have the ability to mix and match them. Let's start by awaiting a promise.all. This will wait until all of the promises that we pass in are complete before continuing. Also remember that async functions return an implicit promise, so let's add our first promise. Let's pause here for a moment because this might be unfamiliar syntax. We're starting by creating an anonymous async function. This function will return a promise, and that will be our first promise. The body of our anonymous function is the familiar async/await call to get our orderStatuses. Next, let's add a second promise, and this time it'll be for our orders call. With this construct, we now have two promises in the array of promise.all, and as we saw in the previous module, promise.all will kick off each of the functions, and wait until they're all complete before returning, and since we're awaiting the promise.all, the parallel function won't end until all of our promises are done. Head over the browser, and reload the page. Now click the parallel button on the screen. Almost instantly you'll see the first order on the screen, then after about one and a half

seconds, the orderStatus will appear. If you check out the console, you'll see that once again, orders finished before orderStatuses, but with this approach, the code was handled as the individual promises settled. So you have an example of how you can combine promise functions with async/await, and that'll actually cause parallel execution to happen.

Summary

With that, you're ready to go out and tackle asynchronous programming in JavaScript. One of the first and most important things that you learned in this course was that every promise has one of three states; pending, when the promise has not yet settled; fulfilled, when the promise returns successfully; and rejected, when the promise has an error or is unsuccessful. We used these three states to kick off requests, handle the return data, or display the errors that came back from the API. You also learned how to create your own promises, and this'll be helpful as you begin to create your own asynchronous code, or wrap existing functions, and turn them into promises. Remember, to create your own promise, you only need to create a new promise object, and then pass in the executor function with a resolve and reject parameter. You can then decide when to use each of those two parameters to update the state of your promise. Then you learned about async/await, and my hope for you is that while you learned some new key words, the concepts were already familiar. At the end of the day, async/await excels at the ability of turning asynchronous code into sequentially executed code. You can make them be concurrent, or even parallel, but one of the best places to use them is when you need your code executed in a particular order. Finally, thank you for taking the time to watch this course. I really enjoyed working through promises and async/await. I know they can be tricky topics; hopefully though, we've been able to demystify the asynchronous concepts. I'd love it if you'd leave a rating for my course, and I'd also love to hear from you. You can reach out to me on Twitter, at the address on the screen.